Simulation Modelling Practice and Theory 110 (2021) 102323

Contents lists available at ScienceDirect

Simulation Modelling Practice and Theory

journal homepage: www.elsevier.com/locate/simpat

Energy-aware workflow task scheduling in clouds with virtual machine consolidation using discrete water wave optimization

Rambabu Medara *, Ravi Shankar Singh, Amit

Department of Computer Science and Engineering, Indian Institute of Technology (BHU), Varanasi, Uttar Pradesh, 221005, India

ARTICLE INFO

Keywords:
Cloud computing
Workflow scheduling
VM consolidation
Water wave optimization
Energy-aware
Resource utilization

ABSTRACT

The scientific workflows are high-level complex applications that demand more computing power. The cloud data center (CDC) remains one of the essential models of economic infrastructure for workflow applications. These CDCs consume a lot of electric power while running workflow applications. Hence, efficient energy-aware scheduling techniques are required to perform the task to a virtual machine (VM) mapping. The existing researches overlooked to join the workflow scheduling and VM consolidation which addresses resource utilization and energy consumption effectively. In this article, we propose an energy-aware algorithm for workflow scheduling in cloud computing with VM consolidation called EASVMC. The proposed EASVMC approach is modeled to address the multi-objectives such as energy consumption, resource utilization, and VM migrations. The EASVMC algorithm runs in two phases task scheduling and VM consolidation (VMC). In the first phase, the task with maximum execution length is mapped to the virtual machine that will perform it with the minimum energy. The second phase contains VM consolidation is a prominent NP-hard problem. The VMC phase categorizes the physical hosts into the normal load, under-loaded and overloaded hosts based on CPU utilization. Double threshold values are used for this purpose. VMs from underloaded and overloaded hosts are migrated to normally loaded hosts. For the VMC phase, we used a nature-inspired meta-heuristic approach called the Water Wave Optimization (WWO) algorithm, which finds a suitable migration plan to reduce the energy consumption by increasing the overall resource utilization and switch off idle hosts after migrating its VMs to a suitable target host. The efficiency of our proposed method evaluated using the WorkflowSim simulation tool with five different real-world scientific workloads. The experimental results show that the EASVMC approach surpassed the similar works in stated objectives irrespective of diverse workloads.

1. Introduction

Over the last few years, the cloud technology [1] has created tremendous potential due to its flexible and pay-as-you-go model. It allows the users to access the cloud services from anywhere, at any time through the Internet [2]. As a growing number of organizations seek to become significant players in today's data-driven economy, cloud data centers remain one of the essential infrastructures. In the cloud data center, virtualization technology has been widely used to virtualize the resources and support the on-demand cloud service paradigm. It provides opportunities for simultaneously running multiple Virtual Machines (VMs) within the infrastructure of a single Physical Machine (PM). There is a massive increase in computing demand in cloud data centers due to the rapid increase in cloud applications. As a result, the cloud data center becomes unsustainable due to the consumption of higher

* Corresponding author.
E-mail addresses: medararambabu.rs.cse18@iitbhu.ac.in (R. Medara), ravi.cse@iitbhu.ac.in (R.S. Singh), amit.student.cse17@iitbhu.ac.in (Amit).

https://doi.org/10.1016/j.simpat.2021.102323
Received 21 December 2020; Received in revised form 12 March 2021; Accepted 26 March 2021
Available online 7 April 2021

energy in placing input requests on large-scale resources. The cloud services are inter-related with the Service Level Agreements (SLAs). Hence, maintaining a trade-off between energy utilization and performance is essential. Several scientific applications such as chemistry, physics, earth science, astronomy, computer science, bioinformatics, etc., contain a large number of precedence constraint tasks [3]. These scientific applications are extremely complex with different sizes of tasks. Usually, these applications are modeled as Directed Acyclic Graph (DAG). The cloud computing environment provides services with different QoS levels to fulfill the workflow application requirements of enormous availability and higher computational energy [4].

Workflow [5] is a set of tasks that are placed in a proper sequence to perform the execution of large-scale real-time applications, which simplifies the processing complexity and management of applications. In DAG, nodes refer to the computational tasks, and edges refer to the precedence constraints between the tasks. To handle the significant increase in the demand for workflow tasks and make sure the minimal energy cost, the service providers necessitate the energy-efficient management of the cloud resources. Hence, the existing works have focused on the optimization approaches that rely on the response time and energy without violating the SLAs. In the cloud computing environment, scheduling the tasks to the resources with the satisfaction of QoS requirements is a critical task, especially for complex applications such as workflows [6]. Recently, several researchers have much attention on effectively utilizing cloud resources and reducing energy consumption. Most notably, the workflow scheduling research works [7,8] have employed the heuristic methods to resolve the constraints in workflow scheduling. Nevertheless, energy-aware workflow scheduling without resource wastage is still in its infancy stage, requiring few efforts to obtain an optimal solution. Thus, this work targets to develop the energy-aware workflow scheduling with VM consolidation without compromising the performance.

The rest of this article is structured as follows. Section 2 discusses the significance of the proposed work and its main contributions. The related work is presented in Section 3. The proposed system models such as the application model, data center model, and energy model are outlined in Section 4. Then Section 5 presents the proposed algorithm implementation. Performance evaluation of the algorithm is explained in Section 6 and finally, Section 7 presents the conclusion and future works.

2. Significance

With the rapid increase in accessing scientific applications, it is required prominence on establishing energy-efficient models. Most of the popular companies such as Google, Apple, Microsoft, and Amazon have utilized cloud services to offer efficient service to the end-user through the cloud computing and storage model. The end-user spending on cloud services is increasing and is expected to 18% growth in the year 2021. Cloud-based conferencing will undergo a 24.3% increase, while cloud-based telephony and messaging will increase at 8.9% clip, according to Gartner, Inc., [9] but its increasing energy cost creates a negative impact on the cloud infrastructure cost. As a result, the rise in energy cost increases the total ownership cost and mitigates the Return On Investment (ROI). The latest studies on the energy utilization of data centers predicted that the total carbon emissions of data centers are about 3.2% of the whole carbon emissions on the globe by 2025. Moreover, the energy consumption not less than one-fifth of worldwide utilization. And by deploying digital data to cloud data centers it is predicted that the carbon emissions might increase to 14 percent of the world's emissions by the year 2040 [10]. Hence, to reduce the annual energy costs and to improve the environment, designing an energy-efficient model is necessary. The more extensive utilization of cloud resources leads to higher energy consumption. Therefore, optimizing the data center resources is a massive need in the real-world without negotiating the performance.

In this work, we address the workflow scheduling problem in cloud environments with objectives to maximize resource utilization, reduce energy consumption and minimize the number of active hosts. We propose an energy-aware approach for workflow task scheduling in clouds with VM consolidation (EASVMC) for cloud schedulers which reduces the utilization of energy of the scientific workflow applications by maximizing the resource utilization and switching off the idle hosts. The proposed EASVMC algorithm implements in two phases VM scheduling to schedule tasks to energy-efficient VMs without compromising the performance and VM consolidation. The VM consolidation phase adapts the WWO algorithm [11] for dynamic VM consolidation. The performance of the proposed EASVMC approach is evaluated by using WorkflowSim [12] toolkit with five different real scientific workflows. The Simulation experiments show that the EASVMC approach can aptly lower the overall energy utilization by maximizing the resource utilization.

The essential contributions of this paper are outlined as follows:

- We present an efficient workflow scheduling algorithm, called EASVMC to reduce the overall energy consumption.
- The proposed algorithm capable of categorizing the physical servers based on CPU utilization and migrate VMs of the overloaded and underloaded hosts to the normal loaded hosts without overloading the target hosts.
- We propose a VM consolidation using a nature-inspired water wave optimization algorithm, which can save energy substantially by maximizing the number of sleeping servers.
- We signify the performance of our approach through different numerical experiments and it shows that the EASVMC algorithm could reach a significant saving in energy consumption, without regard to the diverse workflow structures. Moreover, overall VM migrations are less and maximized sleeping hosts.

3. Related work

Currently, there has been a serious consideration in evolving models for scheduling workflow tasks and energy-aware resource management in clouds. Primarily it depends on diverse factors like user request arrival rate, cloud resources availability, and the workload of tasks.

The VM consolidation problem is extensively studied in cloud data centers. Few recent meta-heuristic works on virtual machine consolidation such as genetic algorithm (GA) [13], ant colony optimization (ACO) [14–17], and particle swarm optimization (PSO) [18,19] based algorithms efficiently reduced energy consumption. A gravitational effect-based VM placement approach in [20] significantly reduced energy consumption by evaluating computer room attributes to carry out the VM deployment. Predictive-based VM placement techniques in [21] saved significant energy by reducing the active PMs while maintaining system performance. A clustering-based VM placement technique proposed in [22] efficiently reduced the execution time but not considered the energy efficiency objective. An ACO-based VM scheduling proposed in [23] uses vector algebra for consolidating the VMs which reduces the energy utilization and maximizes the resource utilization. In our previous work [24] we proposed a shallow water wave-inspired meta-heuristic to efficiently consolidate VMs in the CDC to save energy. However, these algorithms not used scientific workflows for performance evaluation.

Many workflow scheduling approaches in the cloud environment considered optimization parameters such as makespan and cost of the application. Zhou, Xiumin, et al. [25] proposed a fuzzy dominance-based Heterogeneous Earliest Finish Time (HEFT) technique to minimize cost and makespan of workflows in real cloud resources by considering the real pricing models. DAG splitting technique in [26] for large-scale workflow task scheduling reduces the monetary cost by increasing the VM usage. A list-based approach in [27] gives the best makespan. It is observed that most of these works consider bi-objective optimization problems but not addressed the energy objective.

To reduce the energy utilization of the workflow task scheduling in clouds, recent research works have studied different techniques such as resource hibernation, dynamic power management, clustering, etc. The authors in [28] proposed an energy-aware approach for time-constrained workflow in clouds using DVFS (dynamic voltage and frequency scaling) approach. It also considered cost and resource utilization objectives by satisfying SLAs. A Game-Score simulator proposed in [29] for scheduling parallel tasks in the cloud. It trade-off between energy consumption and makespan. Li, Chunlin, et al. [30] proposed workflow scheduling in a distributed cloud which maximizes resource utilization by calculating task running time by considering service status. A PSO-based multi-objective algorithm in [31] minimizes cost, makespan, and maximizes resource utilization while maintaining system reliability. The authors in [32] proposed a PESVMC algorithm that combined the VMC problem with the VM scheduling to schedule the workflow tasks. The authors in [3] proposed reliability and energy-aware approach. Li, Zhongjin, et al. [33] proposed a cost and energy-efficient algorithm which uses sequence and parallel task merging, VM reuse, and slack techniques for saving energy.

In the heterogeneous cloud computing infrastructure, scheduling the workflow applications is a challenging task due to the necessity of satisfying the QoS requirements during the task scheduling process. The conventional workflow scheduling research works have focused on optimizing the time or cost, regardless of energy consumption. The lack of satisfying the QoS requirements has led to SLA violations. In the cloud environment, it is essential to focus on several factors during the task-dependent scientific application execution, including minimizing the response time, the profit maximization, and the energy consumption minimization. The workflow scheduling model often meets the difficulty in controlling the dominance among the multi-objectives in the cloud environment. Even though several research works have focused on providing the non-dominated solutions, it leads to higher computation time due to its increased comparisons over the multiple objectives. During workflow scheduling, finding the optimal solution in a non-preemptive manner is a challenging task when there are a number of parent tasks having the same priority and different workloads in a workflow application. For this context, selecting the earliest-free VM, which completes the execution of any of the parent tasks for its succeeding task, is inefficient. It is because, due to the various workloads, still there are several parent tasks in the running state, which increases the resource wastage and energy consumption until initiating the execution of its succeeding task in the cloud. Hence, it is necessary to effectively manage both the workflow tasks and resources while scheduling on the cloud infrastructure. To ensure the minimized energy consumption in the cloud, the dynamic consolidation of resources has great attention under the consideration of overloaded and underloaded hosts, migratable VM selection, and VM placement after migration.

In the cloud environment, scheduling similar workloads of the tasks in the same PM tends to reduce the performance due to the necessity of similar requirements during execution. Also, the dynamic variation in the resource demand has led to performance degradation with respect to increased response time and computational energy. Hence, it is crucial to focus on VM migration and active resource reduction. In this paper, we propose an energy-efficient algorithm for workflow task scheduling in clouds. Which effectively maps workflow task to VMs and consolidate overloaded and underloaded VMs using water wave-based optimization algorithm.

4. System models

The proposed algorithm in this paper manages the scheduling of the workflow tasks in the cloud environment. It aims to reduce energy consumption, improve resource utilization and minimize the number of VM migrations. To model such a capable approach, we introduced system models considered in this work. We recommend various system models such as the cloud data center, application, and energy model. Table 1 presents the major notations along with descriptions used everywhere in this article for simplicity.

4.1. Data center model

We considered that the data center offers a set of m heterogeneous hosts H $H = \{h_1, h_2, \ldots .., h_m\}$. Every host is described with various types of computing resources like CPUs, memory size, bandwidth, storage, and network capacity. These hosts or PMs are virtualized into several VMs to handle numerous user requests concurrently. The end-users of the cloud are enabled to submit

Table 1
Major notations.

Notation	Definition
H	Set of hosts (PMs)
VM	Set of VMs
H_{normal}	Set of hosts on normal load
H_{under}	Set of hosts on under load
H_{over}	Set of hosts on over load
T	Set of tuples
h_{source}	Source host
h_{dest}	Destination host
vm	VM in a tuple
$vm_k^{opt_i}$	Optimal vm for task t_i in energy consumption
W	Workflow
t_i	ith task
C	Set of edges in DAG
t_{entry}	Entry task
t_{exit}	Exit task
w_i	computation cost of task t_i
$T(t_i, vm_k)$	Execution time of t_i on vm_k
$\overline{T}(t_i)$	Mean execution times of t_i on different available VMs
c_{ij}	Communication edge between t_i and t_j
$w(c_{ij})$	Cost of c_{ij}
$T(t_{ij})$	Communication time between t_i and t_j
T_M	Makespan of W
P_{util}	Power utilization
$P_{CPU_{idle}}$	CPU power utilization at 0% load
$P_{CPU_{full}}$	CPU power utilization at 100% load
h	Wave height
λ	Wave length
λ_{max}	Maximum wave length
$f(\mathbf{x})$	Fitness of a wave $\mathbf{x}$
P_s	Number of sleeping hosts
M	Number of migrations
γ	Parameter to define relative importance of P_s
α	Wavelength reduction coefficient
vm_j^i	jth vm on host h_i
(vm_j^i, t)	Active time of vm_j on h_i
(h_i, t)	Active time of host h_i
RU	Resource utilization (%)
RU_{avg}	Average resource utilization (%)

requests for the provisioning requested number of VMs. Usually, user requests are measured in million instructions (MI). Further, virtual machines are characterized by their computing efficiency in MIPS (Million Instructions Per Second), bandwidth (Bw), etc. Therefore, a cloud scheduler has different capabilities in selecting a reasonable VM to provision the user requests by satisfying the workflow constraints. Note that this article uses notations host and PM interchangeably to represent a cloud server.

4.2. Application model

Usually, workflow applications are modeled as a DAG. A workflow W with a set of n precedence constraint tasks T_W is represents with a DAG $W = (T_w, C)$ where $T_W = \{t_1, t_2, \ldots .t_n\}$ and C is the set of communication edges $(c_{ij} \in C, 1 \leq i, j \geq n \quad and \quad i \neq j)$ that represents the dependencies between tasks. A directed arc or edge c_{ij} $(i, j \in C \quad and \quad i \neq j)$ between the tasks t_i and t_j represents the dependency, where t_i is the parent of t_j. A task having no parent(s) is an entry task t_{entry} and A task without successor(s) is an exit task t_{exit}. Any task t_i is bring on to execution when it provisioned requested computing resources. If a task t_i finishes execution then its output data transfers to all of its successors. The data (in MB) communicated from parent task t_i to its successor task t_j has considered as the cost of edge c_{ij} i.e., $w(c_{ij})$. We express the total running time of W as makespan T_M. The communication cost between any two precedence constraint tasks t_i and t_j is represented as $T(t_{ij})$ and is calculated as in Eq. (1).

$$T(t_{ij}) = \frac{w(c_{ij})}{Bw} \tag{1}$$

where Bw is the bandwidth and $w(c_{ij})$ is the transferred data between t_i and t_j. The effective execution cost of any task t_i on a specific vm is computed using Eq. (2).

$$T(t_i, vm_k) = \frac{w_i}{vm_{mips}^k} + T(t_{ij}) \tag{2}$$

where $T(t_i, vm_k)$ is the effective computing time of task t_i on kth virtual machine vm_k, w_i is the computation cost of task t_i and vm_{mips}^k is the computing capacity of vm_k. For any task t_i a mean computing time is calculated using Eq. (3) and is the average of

execution times of all tasks on a set of virtual machines.

$$\overline{T}(t_i) = \sum_{k=1}^{n} \frac{T(t_i, vm_k)}{n} \tag{3}$$

we calculated the earliest start time EST and earliest finish time EFT of task t_i as follows.

$$EST(t_i) = \begin{cases} 0 & \text{if } t_i = t_{entry} \\ max_{t_p \in parent(t_i)} EFT(t_p) & \text{otherwise} \end{cases} \tag{4}$$

$$EFT(t_i) = EST(t_i) + T(t_i, vm_k) \tag{5}$$

The completion time of the exit task i.e., $EFT(t_{exit})$ is the minimum makespan of the workflow W $minT_M = EFT(t_{exit})$.

4.3. Energy model

The energy dissipation of servers in data centers is primarily because of the CPU, memory, networking devices, storage media, and other underlying circuits. Among these, the CPU dominates energy consumption. The power consumption of a PM in a cloud data center can be precisely specified by a linear relationship with the CPU utilization even by applying DVFS [34]. An active physical server but with zero load utilizes 50% to 70% of the power used by the physical server running at maximum load [35]. Hence, A linear power model proposed in [36] is used in this work. Therefore, the power utilization of a cloud server P_{util} is calculated using the following Eq. (6).

$$P_{util} = P_{CPU_{idle}} + (P_{CPU_{full}} - P_{CPU_{idle}}) * CPU_{util} \tag{6}$$

where $P_{CPU_{full}}$ and $P_{CPU_{idle}}$ are CPU power consumptions at loads 100% and 0% respectively and CPU_{util} is the utilization of the CPU. The energy consumption represents the product of power consumption and time. Therefore, the energy utilization is calculated when the machine is running using Eq. (7).

$$E = P_{util} * t \tag{7}$$

The energy consumption of t_i executing on vm_k is measured using Eq. (8).

$$E(t_i) = P_{vm_k} * T(t_i, vm_i) \tag{8}$$

where P_{vm_k} power consumption of vm_k and $T(t_i, vm_i)$ is the effective run time of t_i on vm_k. The total energy utilized for the workflow W execution is given by the summation of the individual tasks energy utilization in the application.

$$E(W) = \sum_{i=1}^{n} E(t_i) \tag{9}$$

5. Algorithm implementation

The proposed EASVMC algorithm is capable of minimizing energy consumption by effectively applying the workflow scheduling and virtual machine consolidation methods. It is a scheduling application with a bounded number of computing machines, which has been implemented in two phases: task scheduling phase for mapping the workflow tasks to the optimal VMs to minimize energy and VM consolidation phase to maximize the number of sleeping hosts to save more energy. The implementation of the proposed EASVMC algorithm discussed in detail as follows.

5.1. Task scheduling

This phase requires the priorities of tasks based on execution length without disturbing the dependencies among the tasks. The highest priority is assigned to the task with the longest execution length and the task list is generated based on the decreasing priorities. The decreasing priority order shows the topological orders of tasks which preserves task dependencies. The first task t_i is selected from the task list i.e. the task with the highest priority and calculate energy consumed on all available VMs using Eq. (10). The VM, that consumes the least energy is marked as optimal VM vm_k^{opt} for the task t_i and task t_i is mapped to $vm_k^{opt_i}$. The entire procedure for VM selection for the workflow tasks is shown in Algorithm 1.

Algorithm 1: VM Scheduling

```
Compute the mean computation costs of each task using Eq. (3);
Assign tasks priorities based on computational costs;
Prepare a task list for scheduling by decreasing tasks priority order;
while there are unscheduled tasks in the list do
    Select the highest priority task i.e. first task from the task list;
    for each vm_k in the set of VMs do
        Calculate energy consumption of task t_i on vm_k i.e E(t_i, vm_k);
        Map task t_i to the vm_k that minimizes the energy of task t_i;
    end
end
```

For each task, Algorithm 1 needs to traverse on the set of VMs to get the optimal VM which will take minimum energy to execute it. By considering n number of tasks in the workflow W, there are n mappings for resources, one for every task, and for each task, we need to traverse on its parent tasks to get the communication time which can be done in $O(n)$ time. Therefore, the overall time complexity of the VM scheduling algorithms is $O(n^2 v)$, where n is the number of tasks and v is the number of available VMs.

5.2. VM consolidation

This section discusses the VM consolidation problem to maximize sleeping hosts to save energy by improving resource usage of active servers (PMs). The VM Consolidation in cloud data centers is a strict NP-Hard problem [14]. We adopt a new meta-heuristic that is inspired by shallow water wave models called water wave optimization [11] for VM consolidation (WWO-based VM consolidation). It was tested by the CEC2014 benchmark problem and verified that the WWO approach outperforms the state-of-the-art evolutionary approaches namely IWO, BBO, and GSO. Nowadays, the WWO algorithm and its improved versions have been adopting to solve different optimization problems for example flowshop scheduling [37–39], VM consolidation in cloud data centers [24], satellite system design [40], etc. The original WWO is proposed to solve continuous optimization problems. The VM consolidation is a combinatorial problem hence we modified parameters of WWO accordingly to solve discrete optimization problems. The WWO algorithm is efficient in searching for near-optimal solutions in multi-dimensional global optimization problems [24]. The proposed technique dynamically migrates the VMs to improve the host PM resource utilization and reduce energy utilization. It also switches off idle host machines after migrating all VMs from it to a target hosts to save more energy.

We considered a bounded number of host machines (PMs) and virtual machines in this work to carry out the VM consolidation problem. Based on the resource (CPU) utilization the proposed algorithm categorizes the PMs into three groups: over-utilized PMs H_{over}, normal-utilized PMs H_{normal}, and underutilized PMs H_{under}. For any single host, the percentage of resource utilization (RU) is calculated using Eq. (10) and the overall system resource utilization of active hosts using Eq. (11).

$$\%RU_{h_i} = \sum_{j=1}^{k} \frac{vm^{i,j}_{mips}}{h^{i}_{mips} * k} * 100 \tag{10}$$

where $RU(h_i)$ is the resource utilization of the ith physical machine, $vm^{i,j}_{mips}$ is the MIPS acquired by jth vm on host h_i, h^{i}_{mips} is the MIPS of the ith physical machine and k is the total of VMs on host h_i.

$$RU_{avg}(H) = \frac{\sum_{i=1}^{n} RU(h_i)}{n} \tag{11}$$

where $RU_{avg}(H)$ is the average resource utilization for n number of hosts.

We used double (upper and lower) threshold values to categorize PMs based on the percentage of the host resource utilization. All the hosts with resource utilization less than the lower threshold L_{th} are H_{under}, the hosts with resource utilization over the upper threshold U_{th} are H_{over} and the rest of the hosts are H_{normal}. The upper threshold and lower threshold for a host resource utilization are set to 0.9 and 0.5 [41] respectively. The pseudo-code for the process of identifying the type of host is shown in Algorithm 2.

Algorithm 2: Host Categorization

```
for each host h_i do
    Calculate host utilization RU(h_i) using Eq. (10);
    if RU(h_i) < L_th then
        Add host h_i to H_under;
    end
    if RU(h_i) > U_th then
        Add host h_i to H_over;
    else
        Add host h_i to H_normal;
    end
end
```

In terms of VM consolidation in cloud data centers, any PM is a potential source host or destination host. The proposed VM consolidation algorithm prepares a set of three-element tuples T $t = (h_{source}, vm, h_{dest})$ where h_{source} is the source host to migrate the one or more VMs, vm is the specific VM selected on h_{source} for migration and the h_{dest} is the target host to accommodate the VM(s) from h_{source}. While making tuples the algorithm ensures two constraints to restrict generating too many tuples, which improves the time complexity of the algorithm without affecting its performance. The first constraint is the source vm for migration should be not from the normal loaded hosts.

$$h_{source} \in H_{under} \vee h_{source} \in H_{over} \tag{12}$$

where H_{under} and H_{over} are set of underloaded and overloaded hosts respectively. The second constraint to restrict the size of the tuple set further is that the no overloaded host becomes the destination host.

$$h_{dest} \notin H_{over} \tag{13}$$

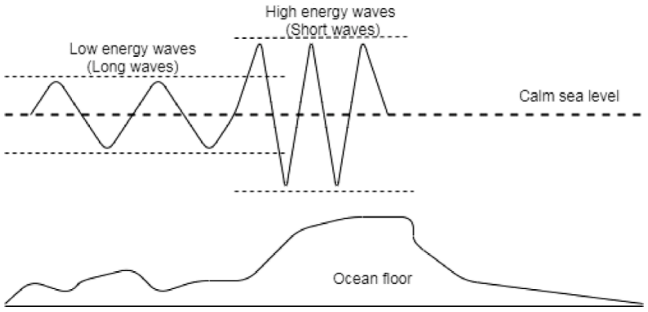

Fig. 1. Shallow and deep water wave models.

The proposed algorithm aims to maximize the number of sleeping hosts by hosting all the VMs on the active hosts without compromising their performance. To accomplish this task, the VM consolidation technique is enforced. Which employ live VM migration to transfer VM(s) between physical hosts to improve the resource utilization and energy efficiency in CDC.

5.3. Water wave based VM consolidation

The shallow water wave-inspired WWO approach is modeled to solve the optimization problems. With a maximizing objective function f and solution space X (which is comparable to the seabed area), the fitness of a point $\mathbf{x}$ ($\mathbf{x} \in X$) is inversely proportional to the seabed area depth. In simple words, the fitness of any point $\mathbf{x}$ $f(\mathbf{x})$ is higher if the point $\mathbf{x}$ is close to the still waters. The variety of shapes of a wave is shown in Fig. 1. The WWO algorithm maintains a set of solutions (population) like other evolutionary algorithms (EAs). Algorithm 3 gives the pseudo-code for the random population (solutions) generation.

Algorithm 3: Population Generation

```
H_source ← H_under ∪ H_over;
H_target ← H_normal;
for each host h_i ∈ H_source do
    for each vm_k on h_i do
        for each host h_j ∈ H_target do
            Add this tuple (h_i, vm_k, h_j) in a possible migration set;
        end
    end
end
while Population size is less than desired do
    x ← EmptyWave;
    Indices ← generate a random permutation of size of all possible set;
    for each indices_i ∈ Indices do
        Add tuple at index as indices_i from possible migration set to x;
    end
    Add this x to population set P;
end
```

The population set P for WWO is generated by the set of possible migration of VMs from over-utilized and underutilized hosts to normal-utilized hosts. The migration from over-utilized hosts to normally utilized hosts balances the load and from underutilized hosts to normally utilized hosts free the hosts then they can be turned off. Each solution is a migration plan and we represented it as a wave $\mathbf{x}$ with three parameters amplitude (height) h, wavelength λ and a tuple t. After the generation of population set P, the search for the best migration plan starts. This search process of the WWO algorithm in the solution space is modeled as the propagation, refraction, and breaking of the waves. Initially, the wavelength and height of all waves in the population set are assigned as 0.5 and h_{max} respectively, and the fitness of each wave $\mathbf{x}$ is calculated using Eq. (14).

$$f(\mathbf{x}) = P_s^{\gamma} + 1/(\epsilon + M) \tag{14}$$

where, P_s is the number of switched off hosts (PMs), M is the number of migrations, γ is a parameter that defines the relative importance of P_s, and ϵ is a small value to avoid division with zero when there are no migrations.

Propagation: During the propagation process, a wave needs to be generated exactly once. If the wavelength λ of current wave $\mathbf{x}$ is less than a randomly generated value r then $\mathbf{x}$ generates exactly one new wave $\mathbf{x}'$. A new tuple t is added to the offspring wave $\mathbf{x}'$, replace the existing tuple with a new tuple, or removing the existing tuple is performed based on two probabilities r_1 and r_2.

These r_1 and r_2 are lower and upper probabilities respectively and are based on series of preliminary experiments to improve the performance of the propagation. If the fitness of $\mathbf{x}'$ is greater than the fitness of $\mathbf{x}$ then replace $\mathbf{x}$ by $\mathbf{x}'$ and set height of $\mathbf{x}$ to h_{max}. Otherwise, the wave $\mathbf{x}$ remains but reduces the height of $\mathbf{x}$ by one. For every generation, the wavelength is updated as follows:

$$\lambda = \lambda \cdot \alpha^{-(f(\mathbf{x})-f_{min}+\epsilon)/(f_{max}-f_{min}+\epsilon)} \tag{15}$$

where, $f(\mathbf{x})$ is fitness function, α is a wavelength reduction coefficient, f_{min} and f_{max} are minimum and maximum fitness of the current solution respectively, and to avoid division with zero in case of no migrations a small values ϵ is used. When fitness is higher the wavelength is less for any wave, hence Eq. (15) ensures propagation in smaller areas.

Refraction: In the theory of a water wave, a wave refracted when its ray is not perpendicular to its isobath. A wave refraction is performed on the wave $\mathbf{x}$ when its height becomes zero i.e., $\mathbf{x}.h = 0$. Suppose $\mathbf{x}*$ is the global best wave so far and the height of a wave $\mathbf{x}$ becomes zero then the wave $\mathbf{x}$ is replaced by new wave $\mathbf{x}'$. The position of the new wave $\mathbf{x}'$ is at the midpoint between the current wave $\mathbf{x}$ and the global best wave $\mathbf{x}*$.

In the context of VM consolidation, every solution is a migration having some tuples representing the solution. When a solution is poor i.e. height h vanishes then add some tuples or replace the existing tuples in the current solution with some tuples from the global best solution. And for every new solution height must be set to h_{max} and then update its wavelength λ' as in Eq. (16).

$$\lambda' = \lambda \cdot \frac{f(\mathbf{x})}{f(\mathbf{x}')} \tag{16}$$

where $f(\mathbf{x})$ and $f(\mathbf{x}')$ are the fitnesses of old and new solutions respectively.

Breaking: Breaking operation performed to find the new best solution if any which will be carried by conducting local search. It is only performed on the wave that gives a better solution than the global best solution. Suppose a newly generated solution $\mathbf{x}$ has better fitness than the global best solution $\mathbf{x}*$ i.e. $f(\mathbf{x}) > f(\mathbf{x}*)$ then $\mathbf{x}$ becomes the new global best solution i.e. $\mathbf{x}* = \mathbf{x}$. Conduct a local search around new global best solution for a better solution than the current $\mathbf{x}*$. If it finds a better solution then the new solution $\mathbf{x}'$ becomes the new global best $\mathbf{x}* = \mathbf{x}'$, otherwise $\mathbf{x}*$ remains the global best solution.

The breaking operation can be performed for a better solution in the following way: After the propagation, if a solution $\mathbf{x}$ found to be better than the $\mathbf{x}*$, then we look for a better solution in the neighborhood of $\mathbf{x}$. For a fixed number of times let say k, select a few tuples (say 5) from the current best solution. Then for each of the selected tuple replace the tuple with a tuple from T excluding the tuples from $\mathbf{x}$. This can produce series of solitary waves, If the solitary wave found to be a better solution $\mathbf{x}'$ than $\mathbf{x}*$ then $\mathbf{x}* = \mathbf{x}'$. The minimum and maximum value for k is 1 and 12 respectively i.e the value of k is a predefined number [11].

Algorithm 4 contains the pseudo-code for the VM consolidation phase. The main objectives of this algorithm are dynamic VM migrations for load balancing and free some hosts to switch off, hence saving energy. It also maximizes the resource utilization of active hosts. In the first two steps, it calls two algorithms a host categorization and population generation. Further, it performs propagation, breaking, and refractions operations for finding the best migration plan.

Algorithm 4: VM Consolidation

```
HOST CATEGORIZATION(H);
POPULATION GENERATION(P);
while stopping criteria not reached do
    for each wave x ∈ P do
        x' ← Perform propagation;
        if f(x') > f(x) and f(x') > f(x*) then
            Perform breaking operation on x';
            Update x, x* ← x';
        else
            Decrement height of x by 1;
        end
        if height of x vanishes then
            Perform refraction operation on x to a new x';
            Update Wavelength using Eq. (15);
        end
    end
end
return Migration Plan;
```

The time complexity of the VM Consolidation phase is $O(h^2v + pw(1 + mw))$ where h is the number of hosts, v is the number of virtual machines, p is the population size, w is the wave size and m is the number of iterations which WWO will perform. To prepare the all possible migration tuples algorithm takes $O(h^2v)$ time, for population generations $O(pw)$ time and to perform the wave operations such as propagation, refraction, and breaking $O(mpw^2)$ time. Table 2 summarizes the functions and time complexity of each sub-algorithm of EASVMC.

Table 2
The EASVMC algorithm summary.

	Maximize resource utilization (%)	Reduce energy	VM Migrations	Time complexity
VM scheduling	–	Yes	–	$O(n^2v)$
Host categorization	–	–	–	$O(h)$
Population generation	–	–	–	$O(h^2v + pw)$
VM consolidation	Yes	Yes	Yes	$O(h^2v + pw(1 + mw))$

Table 3
Workflow benchmark setup.

Workflow	Number of tasks
Montage	50, 75, 100, 125, 150 and 1000
CyberShake	90, 120, 150, 180, 210 and 1000
LIGO	74, 100, 124, 150, 174 and 1000
SIPHT	100, 150, 200, 250, 300 and 1000
Epigenomics	24, 46, 64, 100, 125 and 997

Table 4
VM instance specifications.

VM type	Memory (in GB)	Number of cores	Computing capacity (MIPS)
Micro	1	1	500
Small	2	1	1000
Medium	4	2	2000
Large	8	2	2500

Table 5
WWO parameters.

γ	ϵ	r	r_1	r_2	α	λ	k_{max}
5	0.00001	(0, 1)	.28	.68	(1.001, 1.01)	0.5	12

6. Performance evaluation

We discuss the efficiency of our proposed EASVMC approach in this section. we evaluated the performance of the EASVMC through a sequence of simulation experiments for energy consumption, resource utilization, VM migrations, and the number of switched-off hosts.

6.1. Experimental settings

We used the toolkit called WorkflowSim [12] to simulate the cloud environment, which enables users to simulate the Infrastructure-as-a-Service (IaaS) cloud. The IaaS cloud provides a virtualized platform for scheduling workflow applications. We choose five different workflows such as CyberShake, Montage, SIPHT, LIGO, and Epigenomics from various scientific areas to evaluate the performance of our proposed EASVMC algorithm. For every workflow, we consider five different workloads in the simulation experiments Table 3. Cybershake is a memory-intensive and data-intensive earthquake science application. Montage is an astrophotography application and is characterized as an I/O-intensive astronomy application.

SIPHT is the sRNA Identification Protocol using High throughput Technology (SIPHT) program that is used in bioinformatics applications to empower high-throughput, kingdom-wide prediction, and functional annotation of bacterial sRNA-encoding genes [42]. Epigenomics is the CPU-intensive workflow [43], which is the bioinformatics application that enables the automation of several genome sequencing operations. The Laser Interferometer Gravitational-Wave Observatory (LIGO) is a driving application to detect gravitational waves developed by violent events in the universe. Usually, very largescale datasets are used in scientific workflow applications. Each scientific workflow has its data and computing power requirements and their complete characterization presented by Juve et al. [44]. The topological structures of the five different scientific workflows consider in this paper are as shown in Fig. 2. We have considered four different VM instances in CDC, each VM has its computing resources as given in Table 4 and are self-defined. Each VM type has virtually unlimited instances and a constant Bw among VM instances. Along with ϵ and γ, the other control parameters used in WWO such as maximum wave height, wavelength reduction coefficient α, maximum number of breaking directions k_{max}, wave propagation probabilities r_1 and r_2 for getting good solution are tabulated in Table 5. All these recommended values for these parameters are based on series of experiments to obtain an optimal solution. The smaller values for h_{max} improves the solution diversity by frequently replacing the solution with new solutions and the large value specifies the more life span for the solution. The α values define the size of the area of solution exploration, small values cause intensive explorations.

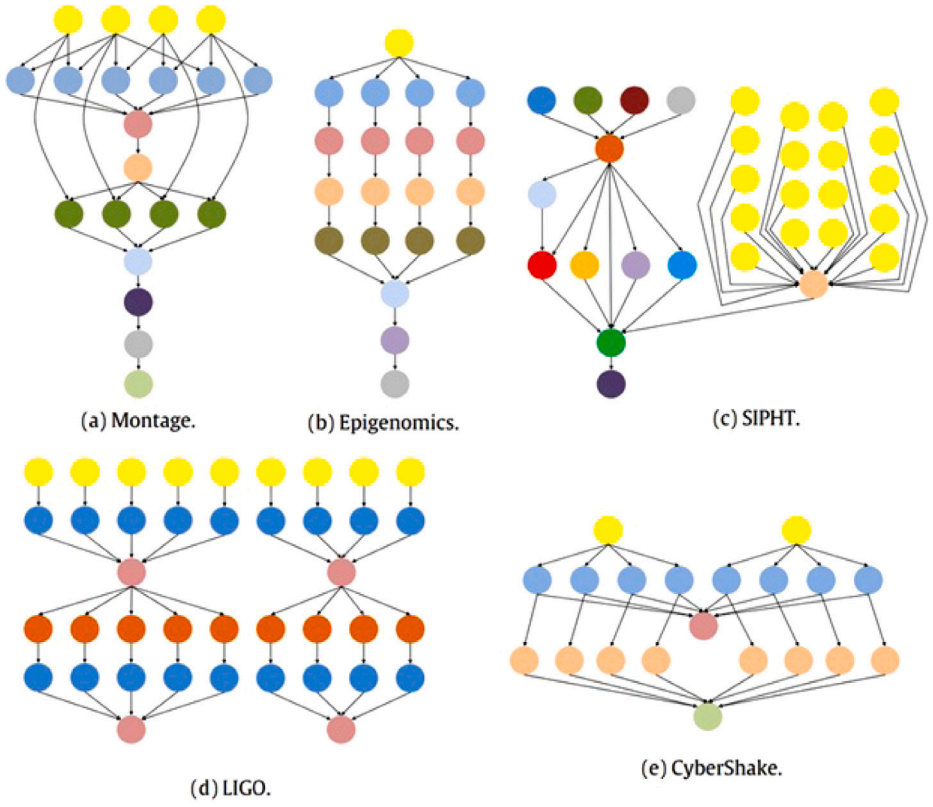

Fig. 2. The structures of five realistic scientific workflows.

6.2. Self-comparison experiments

We conduct self-comparison experiments of energy consumption and resource utilization in this section. To show the energy-saving and resource utilization effect of our sub-algorithms, we use the VM scheduling algorithms as a baseline. It is noted that the VM consolidation algorithm includes two sub-algorithms, i.e., host categorization algorithm and population generation algorithm. The simulation results of self-comparison for energy-saving and maximizing resource utilization are given in Fig. 3, of five scientific workflows. Based on the VM scheduling algorithm, we can see from Fig. 3 that the VM Consolidation algorithm indeed saves more energy and maximizing resource utilization. This is because the VM consolidation algorithm always finds underutilized hosts and switch-off them after migrating its VMs to a suitable target host and migrate VMs from the overloaded hosts for load balancing. Hence it can save more energy and maximize resource utilization.

6.3. Comparison experiments with others

First, we conducted simulation experiments to expose the capability of the VM consolidation EASVMC algorithm in reducing energy consumption and maximizing resource utilization. We perform three well-known VM consolidation algorithms to compare with our VM consolidation algorithm. These algorithms operate the CPU by keeping its load between thresholds. If a host belongs to an underutilized or over-utilized category then the VMs running on such host are migrated to a suitable target host for load balancing. The three reference algorithms are one heuristic algorithm for the dynamic reallocation of VMs in [34] which dynamically sets utilization threshold depend on the Median Absolute Deviation (MAD), and two meta-heuristics ACS-VMC [14], and WWO-VMC [24]. We used parallel workloads in [45] for the simulation experiments. The simulation experimental results for the four algorithms in energy consumption depicted in Fig. 4 for different workloads. Among these VM consolidation policies, our VM consolidation algorithm has the least energy consumption, the MAD performed worst in energy consumption scheduling, and the other two algorithms have medium energy consumption.

Then, we run our EASVMC algorithm for energy consumption, resource utilization, number of vm migrations, and number of switched off hosts. To reveal the strength of the proposed EASVMC scheduling approach, we also perform three popular scheduling approaches HEFT [27], EES (Enhanced Energy-efficient Scheduling) [46] and PESVMC [32] for different objectives. The HEFT and EES (approaches are prominent list-based heuristics for scheduling workflow applications. The power-efficient scheduler with virtual machine consolidation (PESVMC) approach efficiently reduces the energy consumption by switching off the underutilized hosts by

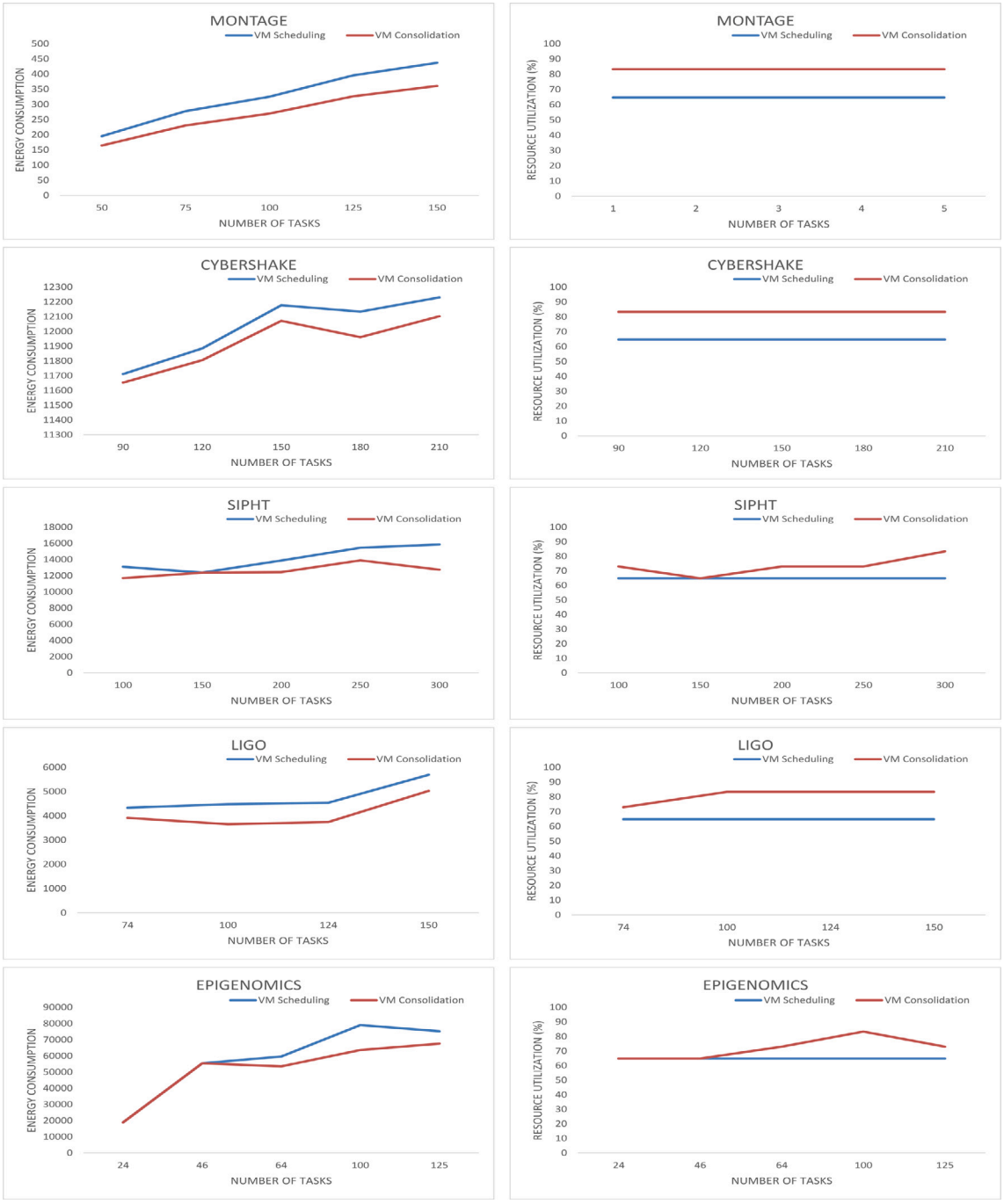

Fig. 3. Self-comparison experiments in energy consumption and resources utilization.

performing VM migrations to a suitable destination host. The PESVMC algorithm identifies the underutilized and over-utilized host machines by using lower and upper utilization threshold values and switch off underutilized host machines by migrating its VMs to normal utilized hosts. The simulation experimental results for energy consumption and resource utilization on five different real-world scientific workflows are depicted in Figs. 5–10.

We conducted several simulation runs for energy consumption and resource utilization on each workflow for different workloads. Our proposed discrete WWO algorithm is an efficient global optimization algorithm that is successful in selecting the optimal target host machines to place the VMs in an energy-efficient way. Hence it saved significant energy compared with other works with different workflows. Further, the resource utilization of the EASVMC algorithm maximized compared with other works. In Figs. 5–9

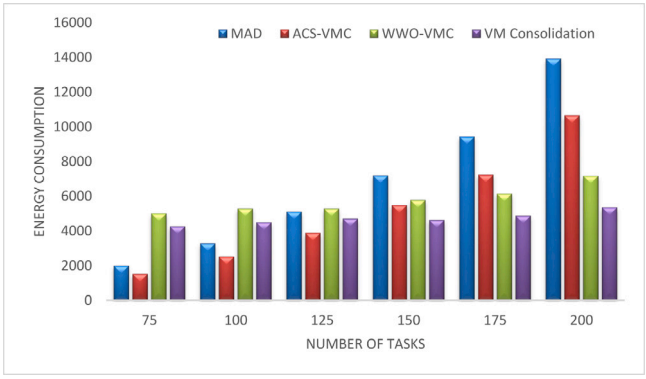

Fig. 4. Comparison experiments of VM consolidation algorithm in energy consumption.

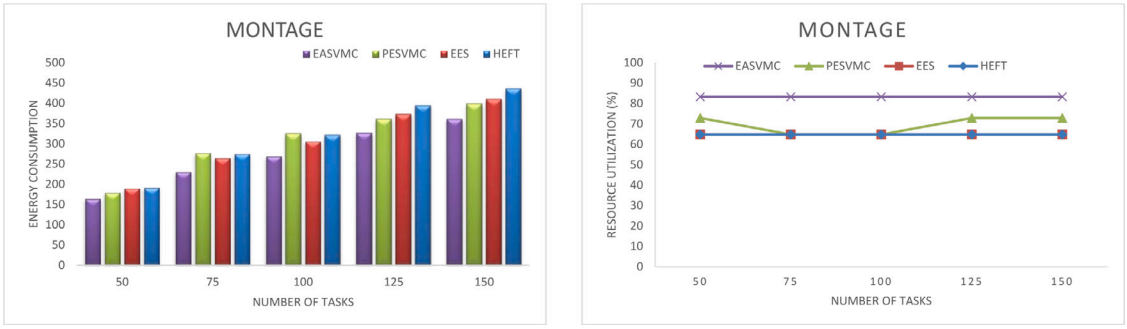

Fig. 5. Energy consumption and Resources utilization on Montage workflow.

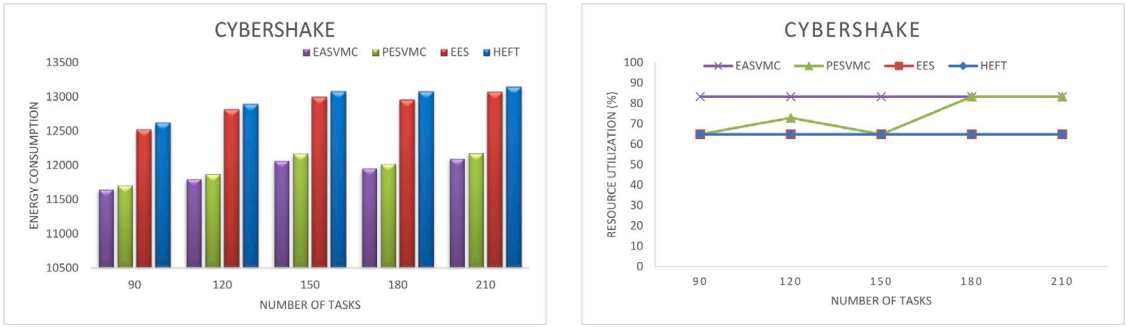

Fig. 6. Energy consumption and Resources utilization on CyberShake workflow.

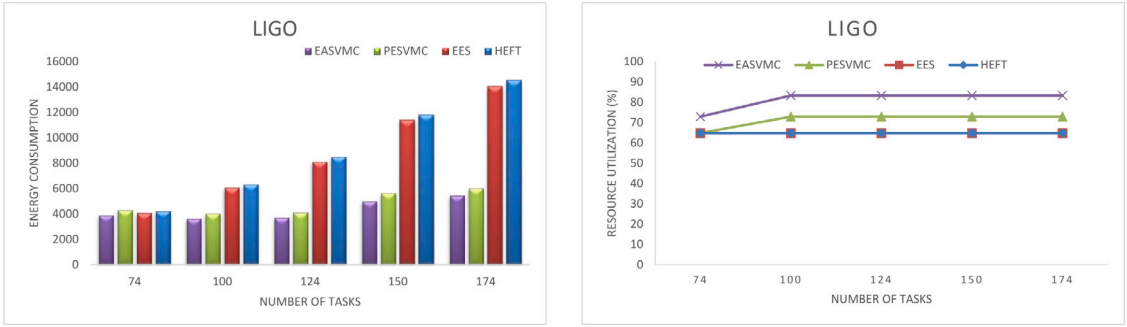

Fig. 7. Energy consumption and Resources utilization on LIGO workflow.

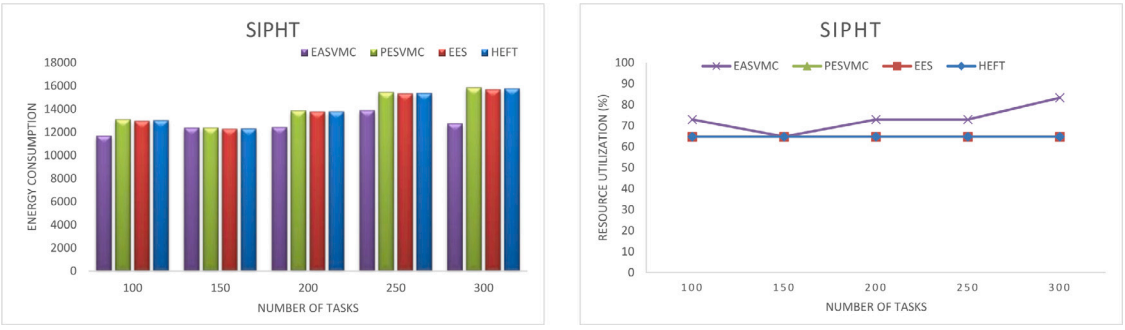

Fig. 8. Energy consumption and Resources utilization on Sipht workflow.

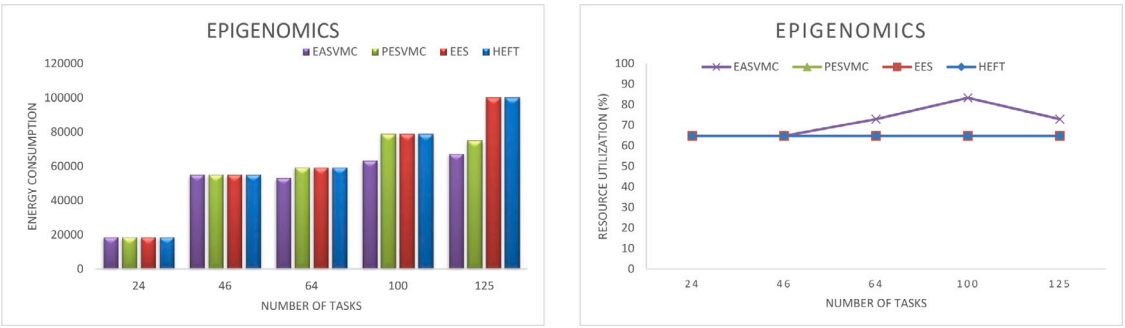

Fig. 9. Energy consumption and Resources utilization on Epigenomics workflow.

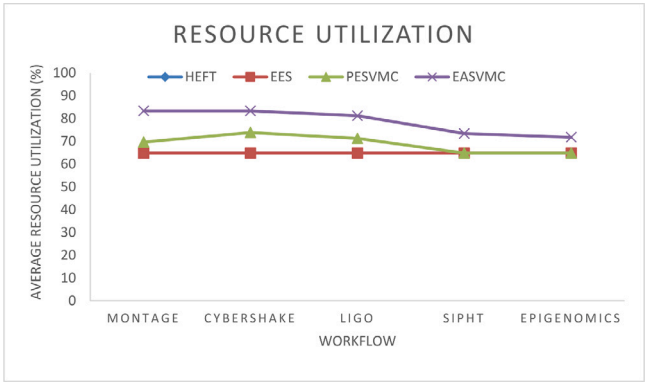

Fig. 10. Average resource utilization on different workflows.

the resource utilization results for HEFT and EES are overlapped. The PESVMC algorithms failed to identify the idle hosts to switch off on both Sipht and Epigenomics workloads in Figs. 8 and 9 respectively. However, consumed less percentage of energy compared with HEFT and EES. This is because of its efficiency in selecting energy-efficient resources to execute tasks. The average resource utilization for the four algorithms depicted in Fig. 10. Results of HEFT and EES are overlapped and the EASVMC outperformed all other three techniques.

The simulation results for the number of *vm* migrations and number of switched off hosts are given in Table 6. The HEFT and EES algorithms have not implemented VM consolidation techniques hence it has zero entries in respective columns. Compared with the PESVMC algorithm our proposed EASVMC algorithm efficient in identifying and switching off idle hosts and reducing overall *vm* migrations. Further, we evaluated our algorithm for large workloads of the five scientific workflows and the results are tabulated in Table 7 for energy consumption, resource utilization, number of *vm* migrations, and number of switched off hosts. In all the cases our proposed EASVMC algorithm surpasses other algorithms. The efficiency of the EASVMC in average energy-saving against the other three algorithms is given in Table 8. In summary, the experimental results reveal that our EASVMC algorithm shows a significant performance advantage over the related well-known techniques in maximizing resource utilization and energy saving irrespective

Table 6
Number of VM migrations and switched off hosts.

Workflow	Number of migrations				Number of switched off hosts			
	HEFT	EES	PESVMC	EASVMC	HEFT	EES	PESVMC	EASVMC
Montage	0	0	7	8	0	0	1	2
	0	0	6	8	0	0	0	2
	0	0	6	8	0	0	0	2
	0	0	6	8	0	0	1	2
	0	0	7	8	0	0	1	2
Epigenomics	0	0	0	0	0	0	0	0
	0	0	1	1	0	0	0	0
	0	0	3	6	0	0	0	1
	0	0	6	7	0	0	0	2
	0	0	4	4	0	0	0	1
SIPHT	0	0	3	6	0	0	0	1
	0	0	2	2	0	0	0	0
	0	0	5	5	0	0	0	1
	0	0	6	6	0	0	0	1
	0	0	6	7	0	0	0	2
LIGO	0	0	3	6	0	0	0	1
	0	0	6	8	0	0	1	2
	0	0	6	9	0	0	1	2
	0	0	6	10	0	0	1	2
	0	0	6	9	0	0	1	2
CyberShake	0	0	6	8	0	0	0	2
	0	0	6	8	0	0	1	2
	0	0	7	7	0	0	0	2
	0	0	9	7	0	0	2	2
	0	0	7	8	0	0	2	2

Table 7
The EASVMC performance for large workloads.

Workflow	Energy consumption				Resource utilization (%)			
	HEFT	EES	PESVMC	EASVMC	HEFT	EES	PESVMC	EASVMC
LIGO-1000	90 641.24	79 231.32071	26 773.19	23 865.87	64.81	64.81	83.33	83.33
SIPHT-1000	21 831.51	20 877.47301	21 994.34	20 497.15	64.81	64.81	83.33	83.33
Montage-1000	2646.25	2527.16875	2474.61	2160.55	64.81	64.81	72.92	83.33
CyberShake-1000	20 241.5	20 160.534	15 889.46	15 518.38	64.81	64.81	72.92	83.33
Epigenomics-997	804 587.1	757 921.067	284 259.2	236 692.8	64.81	64.81	64.81	83.33
	Number of migrations				Number of switched off hosts			
LIGO-1000	0	0	7	7	0	0	2	2
SIPHT-1000	0	0	12	7	0	0	2	2
Montage-1000	0	0	7	7	0	0	1	2
CyberShake-1000	0	0	8	7	0	0	1	2
Epigenomics-997	0	0	5	8	0	0	0	2

Table 8
Energy-saving of the EASVMC algorithm.

Workflow	Energy saving (%)		
	vs. HEFT	vs. EES	vs. PESVMC
Montage	20	15	14
CyberShake	8.1	7.7	.8
LIGO	113	86	9.1
SIPHT	11.5	6.5	12
Epigenomics	21	14	11.5

of diverse workflow structures. This is because of the efficiency of VM scheduling algorithms in selecting the resources and the proposed WWO based VM consolidation algorithm minimizes the number of active hosts and maximizing resource utilization.

7. Conclusion and future work

This work presented the energy-aware scheduling model for the workflow applications, which maintains the trade-off between the performance and energy consumption in the cloud environment. It reviews the existing research works on energy-aware

scheduling in a cloud environment. From the analysis of the conventional research works on the cloud, there is an essential need for performing dynamic and energy-efficient workflow scheduling and ensuring QoS to the end-users. Hence, the proposed approach has modeled the energy-efficient scheduling framework with discrete water wave-based VM consolidation to ensure the reduced energy consumption while assuring the quality of service. The shallow water wave theory inspired WWO approach to perform well in consolidating the VMs to save energy. The algorithm tested for different objectives such as energy consumption, resource utilization, number of *vm* migrations, and number of switched off hosts. We have compared the performance of our algorithm with three well-known approaches HEFT, EES, and PESVMC. The simulation results show that the overall performance of the EASVMC algorithms outperformed the other three approaches.

In future work, we will improve the WWO algorithms for better wave interactions to improve its performance in saving more energy and we will make the algorithm deadline aware which enables the algorithm to take intelligent decisions in selecting VMs for task mapping.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

None. No funding to declare.

References

[1] W. Voorsluys, J. Broberg, R. Buyya, et al., Introduction to cloud computing, in: Cloud Computing: Principles and Paradigms, 2011, pp. 1–44.
[2] M. Rambabu, S. Gupta, R.S. Singh, Data mining in cloud computing: Survey, in: Innovations in Computational Intelligence and Computer Vision, Springer, 2021, pp. 48–56.
[3] R. Medara, R.S. Singh, Energy efficient and reliability aware workflow task scheduling in cloud environment, Wirel. Pers. Commun. (2021) 1–20.
[4] R. Buyya, A. Beloglazov, J. Abawajy, Energy-efficient management of data center resources for cloud computing: A vision, architectural elements, and open challenges, in: PDPTA 2010: Proceedings of the 2010 International Conference on Parallel and Distributed Processing Techniques and Applications (Las Vegas NV, July 12-15, 2010), 2010, pp. 6–17.
[5] E. Deelman, D. Gannon, M. Shields, I. Taylor, Workflows and e-science: An overview of workflow system features and capabilities, Future Gener. Comput. Syst. 25 (2009) 528–540.
[6] F. Fakhfakh, H.H. Kacem, A.H. Kacem, Workflow scheduling in cloud computing: a survey, in: 2014 IEEE 18th International Enterprise Distributed Object Computing Conference Workshops and Demonstrations, IEEE, 2014, pp. 372–378.
[7] L. Liu, M. Zhang, Y. Lin, L. Qin, A survey on workflow management and scheduling in cloud computing, in: 2014 14th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, IEEE, 2014, pp. 837–846.
[8] L. Singh, S. Singh, A survey of workflow scheduling algorithms and research issues, Int. J. Comput. Appl. 74 (15).
[9] J. Moore, Gartner forecasts near 20% growth for public cloud services, 2020, URL https://searchitchannel.techtarget.com/news/252483250/Gartner-forecasts-near-20-growth-for-public-cloud-services.
[10] C. Trueman, Why data centers are the new frontier in the fight against climate change, 2020, URL https://www.computerworld.com/article/3431148/why-data-centres-are-the-new-frontier-in-the-fight-against-climate-change.html.
[11] Y.-J. Zheng, Water wave optimization: a new nature-inspired metaheuristic, Comput. Oper. Res. 55 (2015) 1–11.
[12] W. Chen, E. Deelman, Workflowsim: A toolkit for simulating scientific workflows in distributed environments, in: 2012 IEEE 8th International Conference on E-Science, IEEE, 2012, pp. 1–8.
[13] S. Farzai, M.H. Shirvani, M. Rabbani, Multi-objective communication-aware optimization for virtual machine placement in cloud datacenters, Sustain. Comput.: Inform. Syst. (2020) 100374.
[14] F. Farahnakian, A. Ashraf, T. Pahikkala, P. Liljeberg, J. Plosila, I. Porres, H. Tenhunen, Using ant colony system to consolidate VMs for green cloud computing, IEEE Trans. Serv. Comput. 8 (2015) 187–198, http://dx.doi.org/10.1109/TSC.2014.2382555.
[15] M.-H. Malekloo, N. Kara, M. El Barachi, An energy efficient and sla compliant approach for resource allocation and consolidation in cloud computing environments, Sustain. Comput.: Inform. Syst. 17 (2018) 9–24.
[16] T. Shabeera, S.M. Kumar, S.M. Salam, K.M. Krishnan, Optimizing vm allocation and data placement for data-intensive applications in cloud using aco metaheuristic algorithm, Eng. Sci. Technol., Int. J. 20 (2017) 616–628.
[17] A. Ragmani, A. Elomri, N. Abghour, K. Moussaid, M. Rida, Faco: A hybrid fuzzy ant colony optimization algorithm for virtual machine scheduling in high-performance cloud computing, J. Ambient Intell. Humaniz. Comput. (2019) 1–13.
[18] V.D. Reddy, G. Gangadharan, G.S.V. Rao, Energy-aware virtual machine allocation and selection in cloud data centers, Soft Comput. 23 (2019) 1917–1932.
[19] J. Yan, H. Zhang, H. Xu, Z. Zhang, Discrete pso-based workload optimization in virtual machine placement, Pers. Ubiquitous Comput. 22 (2018) 589–596.
[20] X. Xu, Q. Zhang, S. Maneas, S. Sotiriadis, C. Gavan, N. Bessis, Vmsage: a virtual machine scheduling algorithm based on the gravitational effect for green cloud computing, Simul. Model. Pract. Theory 93 (2019) 87–103.
[21] R. Shaw, E. Howley, E. Barrett, An energy efficient anti-correlated virtual machine placement algorithm using resource usage predictions, Simul. Model. Pract. Theory 93 (2019) 322–342.
[22] J. Zhang, X. Wang, H. Huang, S. Chen, Clustering based virtual machines placement in distributed cloud computing, Future Gener. Comput. Syst. 66 (2017) 1–10.
[23] M.H. Ferdaus, M. Murshed, R.N. Calheiros, R. Buyya, Virtual machine consolidation in cloud data centers using aco metaheuristic, in: European Conference on Parallel Processing, Springer, 2014, pp. 306–317.
[24] R. Medara, R.S. Singh, U.S. Kumar, S. Barfa, Energy efficient virtual machine consolidation using water wave optimization, in: 2020 IEEE Congress on Evolutionary Computation (CEC), IEEE, 2020, pp. 1–7.
[25] X. Zhou, G. Zhang, J. Sun, J. Zhou, T. Wei, S. Hu, Minimizing cost and makespan for workflow scheduling in cloud using fuzzy dominance sort based heft, Future Gener. Comput. Syst. 93 (2019) 278–289.
[26] H. Wu, X. Chen, X. Song, C. Zhang, H. Guo, Scheduling large-scale scientific workflow on virtual machines with different numbers of vcpus, J. Supercomput. (2020) 1–32.

[27] H. Topcuoglu, S. Hariri, M.-y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, IEEE Trans. Parallel Distrib. Syst. 13 (2002) 260–274.
[28] M. Safari, R. Khorsand, Energy-aware scheduling algorithm for time-constrained workflow tasks in dvfs-enabled cloud environment, Simul. Model. Pract. Theory 87 (2018) 311–326.
[29] D. Fernández-Cerero, A. Jakóbik, A. Fernández-Montes, J. Kołodziej, Game-score: Game-based energy-aware cloud scheduler and simulator for computational clouds, Simul. Model. Pract. Theory 93 (2019) 3–20.
[30] C. Li, J. Tang, T. Ma, X. Yang, Y. Luo, Load balance based workflow job scheduling algorithm in distributed cloud, J. Netw. Comput. Appl. 152 (2020) 102518.
[31] F. Abazari, M. Analoui, H. Takabi, S. Fu, Mows: multi-objective workflow scheduling in cloud computing based on heuristic algorithm, Simul. Model. Pract. Theory 93 (2019) 119–132.
[32] N. Mohanapriya, G. Kousalya, P. Balakrishnan, C. Pethuru Raj, Energy efficient workflow scheduling with virtual machine consolidation for green cloud computing, J. Intell. Fuzzy Systems 34 (2018) 1561–1572.
[33] Z. Li, J. Ge, H. Hu, W. Song, H. Hu, B. Luo, Cost and energy aware scheduling algorithm for scientific workflows with deadline constraint in clouds, IEEE Trans. Serv. Comput. 11 (2018) 713–726, http://dx.doi.org/10.1109/TSC.2015.2466545.
[34] A. Beloglazov, R. Buyya, Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers, Concurr. Comput.: Pract. Exper. 24 (2012) 1397–1420.
[35] X. Liu, Z. Zhan, J.D. Deng, Y. Li, T. Gu, J. Zhang, An energy efficient ant colony system for virtual machine placement in cloud computing, IEEE Trans. Evol. Comput. 22 (2018) 113–128, http://dx.doi.org/10.1109/TEVC.2016.2623803.
[36] T. Guérout, T. Monteil, G. Da Costa, R.N. Calheiros, R. Buyya, M. Alexandru, Energy-aware simulation with dvfs, Simul. Model. Pract. Theory 39 (2013) 76–91.
[37] Z. Shao, D. Pi, W. Shao, A novel multi-objective discrete water wave optimization for solving multi-objective blocking flow-shop scheduling problem, Knowl.-Based Syst. 165 (2019) 110–131.
[38] F. Zhao, L. Zhang, H. Liu, Y. Zhang, W. Ma, C. Zhang, H. Song, An improved water wave optimization algorithm with the single wave mechanism for the no-wait flow-shop scheduling problem, Eng. Optim. 51 (2019) 1727–1742.
[39] F. Zhao, L. Zhang, Y. Zhang, W. Ma, C. Zhang, H. Song, A hybrid discrete water wave optimization algorithm for the no-idle flowshop scheduling problem with total tardiness criterion, Expert Syst. Appl. 146 (2020) 113166.
[40] Y. Jin, S. Li, L. Ren, A new water wave optimization algorithm for satellite stability, Chaos Solitons Fractals 138 (2020) 109793.
[41] H. Choi, J. Lim, H. Yu, E. Lee, Task classification based energy-aware consolidation in clouds, Sci. Program. (2016).
[42] J. Livny, H. Teonadi, M. Livny, M.K. Waldor, High-throughput, kingdom-wide prediction and annotation of bacterial non-coding rnas, PLoS One 3 (2008) e3197.
[43] P.W. Laird, Institutional profile: the usc epigenome center, Epigenomics 1 (2009) 29–31.
[44] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, K. Vahi, Characterizing and profiling scientific workflows, Future Gener. Comput. Syst. 29 (2013) 682–692.
[45] D.G. Feitelson, B. Nitzberg, Job characteristics of a production parallel scientific workload on the nasa ames ipsc/860, in: Workshop on Job Scheduling Strategies for Parallel Processing, Springer, 1995, pp. 337–360.
[46] Q. Huang, S. Su, J. Li, P. Xu, K. Shuang, X. Huang, Enhanced energy-efficient scheduling for parallel applications in cloud, in: 2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (Ccgrid 2012), IEEE, 2012, pp. 781–786.